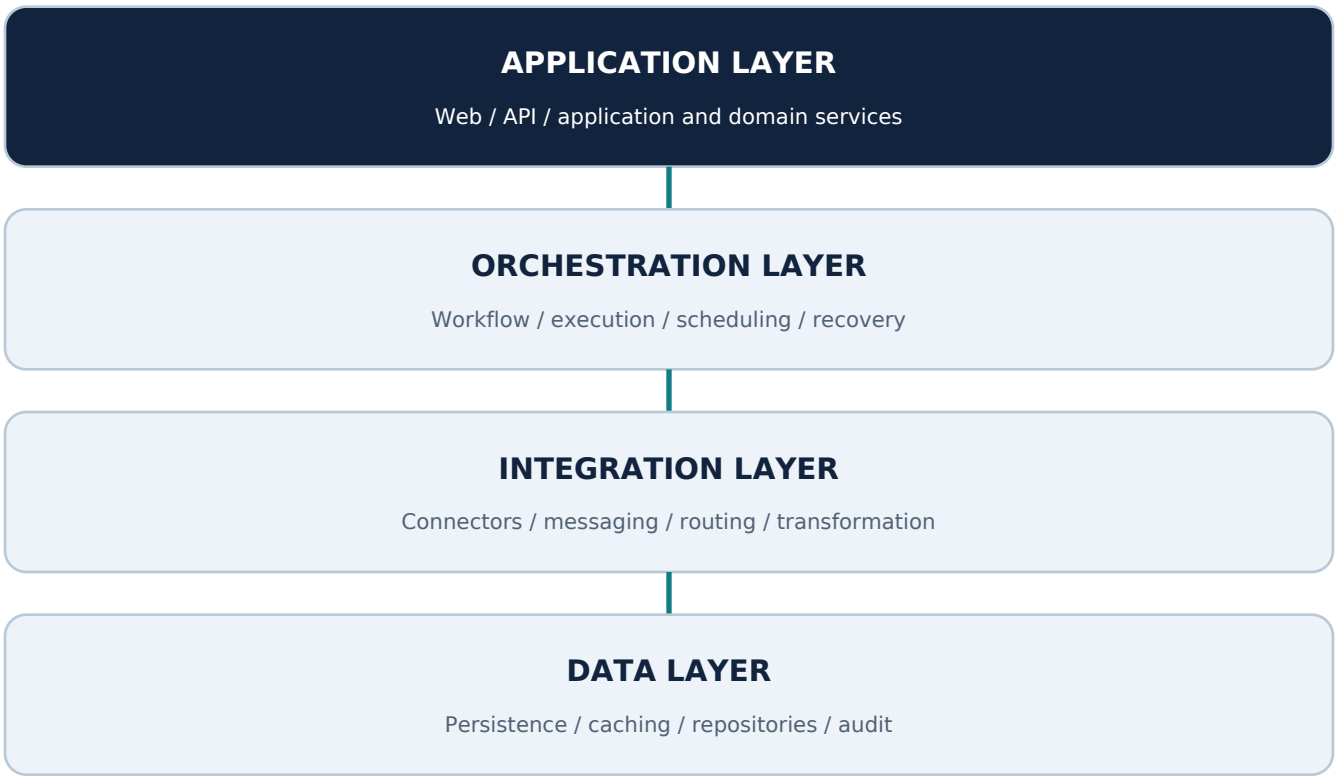
ENTERPRISE PLATFORM ARCHITECTURE

Version 4.0 | Architecture design brief | October 27, 2026

A configurable enterprise application and integration platform organized into four functional layers, a shared Core Layer, and a separate Control Plane.

Architecture at a glance



Core Layer: shared security, configuration, observability, resilience, contracts, execution context, extension framework, and hosting.

Control Plane: separate administrative and management capability for workflow definitions, connectors, schemas, runtime configuration, and operations.

1. Functional layer responsibilities

Layer	Primary responsibility	Representative capabilities
Application	Present capabilities to users and consumers.	Web apps, APIs, request handling, application services, domain services
Orchestration	Coordinate multi-step processing and durable execution.	Workflow engine, scheduling, parallel execution, state, retries, compensation
Integration	Communicate with external systems and transport data.	HTTP/SOAP, messaging, files/SFTP, adapters, event routing, transformation
Data	Own persistence mechanisms and data access.	SQL/NoSQL, repositories, cache, transaction boundaries, audit and message stores

2. Shared Core Layer

Core is a foundational capability set, not a required stop in the transaction path. Core abstractions should remain independent of the functional layers. Infrastructure implementations are registered through dependency injection.

Capability	Scope
Configuration	Typed settings, validation, environment overrides, safe refresh, feature flags
Security	Authentication and authorization abstractions, policy enforcement, secrets references
Observability	Structured logs, distributed traces, metrics, correlation IDs, health checks
Resilience	Timeouts, retries, circuit breakers, bulkheads, idempotency guidance
Contracts	Versioned requests, responses, events, error models and schema conventions
Execution context	Identity, correlation, tenant metadata, cancellation, execution identifiers
Extensions and hosting	Approved modules, startup registration, lifecycle and graceful shutdown

3. Control Plane

The Control Plane manages configuration and governance, while the runtime processes transactions using published, validated definitions. Runtime execution should continue using the last known good configuration if management services are unavailable.

Service	Responsibilities
Platform Administration	Roles, environment administration, permissions and tenant policies
Workflow Management	Create, validate, version, publish, retire and roll back workflow definitions
Integration Management	Connector registry, endpoint definitions, routing, credential references
Schema and Contract Management	Schema versions, validation and compatibility policies
Runtime Configuration	Feature flags, limits, policies and controlled configuration rollout
Operations Management	Execution visibility, failed-run triage, retry controls and audit history

4. Execution and management paths

Management path

Administrator → Control Plane → Validate → Version → Approve/Publish → Runtime configuration distribution

Execution path

Consumer → Application → Orchestration → Integration → Data

The execution path represents the primary conceptual flow, not a mandatory call chain. For example, an application may read directly from an approved data service, and an integration event may trigger orchestration without an interactive application request. All cross-layer access should use explicit contracts.

5. Configuration-driven workflow example

An administrator defines a workflow with approved step types. The Control Plane validates and publishes an immutable version; Orchestration resolves registered handlers at runtime.

```
{
"workflowId": "customer-enrichment",
"version": 1,
"steps": [
{ "id": "validate", "type": "validation", "next": "enrich" },
{ "id": "enrich", "type": "integration", "connector": "customer-service", "next": "persist" },
{ "id": "persist", "type": "data", "operation": "save-customer" }
]
}
```

6. Architectural guardrails

- Core must not reference Application, Orchestration, Integration, or Data implementations.
- Published workflow and connector configurations are immutable, versioned and auditable.
- Runtime execution must not synchronously depend on Control Plane availability.
- Retries require idempotency and must respect transaction boundaries.
- Business rules belong to application/domain services; orchestration coordinates their execution.
- Sensitive configuration stores references to secrets, not plaintext credentials.
- Plugins are allowlisted and registered at startup initially; runtime code loading is not assumed.
- Distributed traces and standardized errors should follow operations end to end.

7. Proposed .NET solution organization

Area	Logical projects / modules
Core	Abstractions, Configuration, Security, Observability, Resilience, Hosting
Application	Api, Web, Services
Orchestration	Engine, Workflows, Execution, Scheduling
Integration	Abstractions, Http, Messaging, Connectors
Data	Abstractions, SqlServer, Caching, Persistence
ControlPlane	Api, Web, Workflows, Integrations, Schemas, Configuration, Operations
Tests	Core, Application, Orchestration, Integration and Data tests

These are logical boundaries rather than a mandate for one assembly per module. Begin with cohesive deployable units and split only where ownership, dependency isolation, scalability, or release independence justify it.

8. Platform requirements

Requirement	Architectural decision
Versioned configuration	Immutable published definitions and explicit rollback
Safe publishing	Validation, authorization, audit and atomic activation
Runtime independence	Last-known-good configuration and resilient distribution
Security separation	Separate administrative and execution permissions
Operational visibility	Track executions, errors, retries, duration and service health
Flexible processing	Support synchronous, asynchronous, scheduled and event-driven modes

9. Next design milestones

- Execution model: sync/async jobs, durable workflows, concurrency, cancellation and compensation.
- Messaging architecture: broker choice, event contracts, outbox/inbox, dead-letter and replay.
- Data architecture: ownership, transactional consistency, schema evolution and retention.
- Deployment architecture: hosting, topology, scaling, security zones and disaster recovery.
- Control Plane design: publishing workflow, RBAC, registry schema and runtime distribution.

Design status: Conceptual architecture approved for further detailed design; implementation and infrastructure choices remain open.